

Amelioration du Socle Django

Guide explique pas a pas pour debutant

Projet Qwen3-ASR - application de transcription audio vers texte

Document genere le 14 septembre 2026

Contenu : modeles de donnees, migrations, upload valide, API de depot et de consultation

. Sommaire

1	Introduction : le projet et la demande
2	Les bases de Django (pour bien comprendre)
3	Le modele de donnees : TranscriptionJob
4	La validation des fichiers uploades
5	L'API : depots et consultation
6	La migration de base de donnees
7	L'interface d'administration
8	Les tests automatiques
9	Mettre en pratique : serveur, curl, interface
10	Resume et prochaines etapes
Annexe	Glossaire du debutant

1. Introduction : le projet et la demande

Le projet

Qwen3-ASR est une petite application web construite avec Django. Son but : permettre a un utilisateur de déposer un fichier audio (une interview, un enregistrement, une reunion...) et de recuperer la transcription écrite de ce fichier, c'est-a-dire le texte de ce qui est dit. Cette transcription est produite par un modele d'intelligence artificielle appele Qwen3 (fichier `qwen_service.py` dans le projet).

La demande

La demande etait : "relancer le projet en ameliorant le socle Django : modeles de donnees, migrations, upload valide, API de depot et de consultation afin qu'un fichier puisse y etre depose et stocke". En clair, il fallait construire la fondation solide de l'application :

- Des modeles de donnees propres : la definition des objets que l'on stocke (ici, les taches de transcription).
- Des migrations : le mecanisme qui modifie la base de donnees sans rien casser.
- Un upload valide : refuser les fichiers qui ne sont pas des fichiers audio, trop lourds ou dangereux.
- Une API de depot et de consultation : des adresses (URL) qui permettent a un programme ou a une page web de depoter un fichier et de suivre son traitement.
- Le stockage du fichier : l'audio est enregistre sur le disque avec un nom unique.

Ce qui a ete fait, en une phrase

On a ameliore le modele de donnees (2 nouveaux champs, stockage avec nom unique), cree un module de validation des fichiers, ajoute une vraie API JSON pour deposer/consulter/supprimer des fichiers, cree une migration, enregistre le modele dans l'administration, et ecrit 37 tests automatiques pour verifier que tout fonctionne. Tout est vert : 37 tests sur 37 passent.

2. Les bases de Django (pour bien comprendre)

Django est un "framework" : un ensemble d'outils tout prepares qui evite de reinventer la roue pour creer des sites web en Python. Son fonctionnement repose sur 4 briques principales, que l'on retrouve dans notre projet :

Le modele (models.py)

C'est la definition des donnees. Chaque classe Python = une table dans la base de donnees, chaque attribut = une colonne. Exemple : la classe TranscriptionJob represente une tache de transcription, avec des colonnes comme original_name (le nom du fichier depose) ou status (en cours, termine, en erreur...).

La vue (views.py)

C'est le code qui recoit une requete (ex: "je veux deposer ce fichier") et repond (ex: "tres bien, voici l'identifiant de votre tache"). C'est le cerveau de l'application.

L'URL (urls.py)

C'est l'adresse. Un fichier urls.py fait le lien entre une adresse web et une vue. Exemple : l'adresse /api/status/12345/ affiche l'etat de la tache 12345.

Le template (templates/)

C'est la page HTML qui s'affiche dans le navigateur. Notre projet en contient plusieurs (dashboard, upload, liste des taches...).

Le cycle d'une requete, image simple

Navigateur (ou programme) -> URL -> Vue -> Modele (base de donnees) -> Reponse -> Affichage.

3. Le modele de donnees : TranscriptionJob

Tout est dans le fichier transcriptions/models.py. Une TranscriptionJob est une tache de transcription : elle se souvient du fichier audio, de son etat, et du resultat. Voici les champs principaux :

Champ	Type	Role
id	UUID	Identifiant unique de la tache (ex: 3f2a...) - genere automatiquement
audio_file	FileField	Le fichier audio lui-meme, stocke sur le disque dans uploads/
original_name	CharField	Le nom original du fichier depose (ex: interview.mp3)
file_size	IntegerField	NOUVEAU - Taille du fichier en octets (ex: 5242880 = 5 Mo)
content_type	CharField	NOUVEAU - Type MIME annonce (ex: audio/mpeg pour un mp3)
prompt	TextField	Instructions optionnelles donnees au modele IA
language	CharField	Langue demandee (vide = detection automatique)
max_new_tokens	IntegerField	Longueur max du texte genere (default 512)
status	CharField	Etat : pending, running, completed, failed
error	TextField	Message d'erreur si le traitement a echoue
segments_json	JSONField	Resultat decoupe en segments avec horodatage
duration_sec	FloatField	Duree du fichier audio en secondes
created_at	DateTimeField	Date de depot (remplie automatiquement)

Les nouveaux champs : file_size et content_type

Avant, on ne savait pas combien pesait le fichier ni de quel type il etait. Maintenant, l'API enregistre ces informations au moment du depot. Utilitaires :

- Afficher la taille de chaque fichier dans le tableau de bord ou l'administration.
- Verifier qu'un fichier est bien un audio avant de le traiter (cote serveur).
- Calculer l'espace disque total utilise (endpoint /api/stats/).

Pourquoi un nom de fichier avec UUID ?

Lors du depot, le fichier est renomme en uploads/3f2a9b1e_mon_interview.wav au lieu de garder mon_interview.wav. L'UUID est un identifiant aleatoire quasi-unique. Cela evite deux problemes :

- Deux personnes deposant interview.wav peuvent avoir chacune leur fichier, sans ecrasement.
- Un nom de fichier bizarre ou dangereux (avec espaces, caracteres etrangers...) est neutralise.

Les statuts d'une tache

Statut	Signification
pending	Le fichier est bien reçu et en attente de traitement
running	L'IA est en train de transcrire
completed	Transcription terminée, le texte est disponible
failed	Une erreur est survenue (fichier illisible, modele indisponible...)

to_dict() : ce que l'API renvoie

La methode to_dict() transforme la tache en dictionnaire JSON, le format lisible par n'importe quel programme. Voici un exemple de reponse :

```
{
  "id": "3f2a9b1e-...",
  "original_name": "interview.wav",
  "file_size": 5242880,
  "content_type": "audio/wav",
  "status": "pending",
  "created_at": "2026-09-14T10:30:00Z",
  "error": null,
  ...
}
```

4. La validation des fichiers uploades

Un de nos emplois accepte n'importe quel fichier. Notre application a besoin de fichiers audio/video. Le nouveau module transcriptions/upload_validation.py refoule tout ce qui n'est pas acceptable, avec des messages clairs en francais. Trois controles successifs :

1) L'extension du fichier

On regarde la fin du nom : .wav, .mp3, .ogg... Les extensions autorisees :

```
wav, mp3, mp4, flac, m4a, ogg, opus, aac, wma, mov, mkv, webm, amr
```

Un fichier document.txt est donc refuse : "Extension '.txt' non prise en charge."

2) Le type MIME

Chaque fichier porte une etiquette technique appelee type MIME, qui dit ce qu'il est (audio/mpeg pour un mp3, video/mp4 pour une video...). On accepte les familles audio/* et video/*. Les navigateurs n'etiquettent pas toujours bien les fichiers .ogg ou .opus : on tolere donc aussi application/octet-stream quand l'extension est valide.

3) La taille

Un fichier de 50 Go ferait planter le serveur. La limite est fixee a 2000 Mo (2 Go) par default, modifiable dans les reglages (settings.py, variable MAX_UPLOAD_SIZE_MB). Au-dela : "Fichier trop volumineux : 3500.0 Mo, maximum autorise 2000 Mo."

Ou la validation est-elle appelee ?

Dans la vue api_job_create (views.py), a chaque depot. Si un controle echoue, la reponse est un code HTTP 400 (requete invalide) avec le message d'erreur en JSON. Le fichier n'est jamais enregistre sur le disque : inutile de stocker un fichier refuse.

A retenir : La validation se fait dans l'ordre : extension, puis type MIME, puis taille. La premiere erreur rencontree bloque tout le reste.

5. L'API : depot et consultation

Une API (Interface de Programmation) est un ensemble d'adresses web que les programmes peuvent utiliser, exactement comme toi quand tu cliques sur un lien. Notre API parle JSON, le format standard d'echange de donnees. Voici les 5 adresses disponibles :

Methode	Adresse	Action
POST	/jobs/create/	Deposer un fichier audio (cree une tache)
GET	/api/status/<id>/	Consulter l'etat d'une tache
GET	/api/jobs/	Lister les taches (avec filtres)
GET	/api/stats/	Statistiques globales
POST	/jobs/<id>/delete-api/	Supprimer une tache (et son fichier)

<id> est l'identifiant UUID de la tache, recu lorsqu'on cree la tache.

Deposer un fichier (POST /jobs/create/)

C'est l'action centrale : elle repond a la demande "un fichier puisse etre depose et stocke". On envoie le fichier dans le champ audio_file. Exemple avec l'outil curl :

```
curl -F "audio_file=@interview.wav" http://127.0.0.1:8001/jobs/create/
```

Reponse attendue : code 201 (cree) avec le detail de la tache en JSON, dont son id. Derriere les coulisses, la vue a :

1. recupere le fichier envoye ;
2. valide extension + type MIME + taille ;
3. cree la tache en base de donnees ;
4. stocke le fichier sur le disque sous uploads/<uuid>_<nom> ;
5. renvoie le JSON de la tache.

Consulter l'etat d'une tache (GET /api/status/<id>/)

Une fois le fichier depose, on peut suivre l'avancement :

```
curl http://127.0.0.1:8001/api/status/3f2a9b1e-.../
```

Reponse : le JSON complet de la tache avec son status (pending, running, completed ou failed), l'erreur s'il y en a une, et le resultat quand il est pret.

Lister les taches (GET /api/jobs/)

Renvoie une liste, avec des filtres pratiques :

```
curl http://127.0.0.1:8001/api/jobs/  
curl "http://127.0.0.1:8001/api/jobs/?status=completed"  
curl "http://127.0.0.1:8001/api/jobs/?limit=10"
```

Statistiques (GET /api/stats/)

Compteurs globaux : nombre total de taches, taches terminees, en erreur, secondes d'audio traitees, espace disque occupe...

Supprimer (POST /jobs/<id>/delete-api/)

Supprime la tache en base ET le fichier audio sur le disque. Utile pour faire le menage.

Les codes de reponse, a connaitre

Code	Signification	Exemple d'usage
201	Cree avec succes	Un fichier vient d'etre depose
200	Tout va bien	Lecture de l'etat d'une tache
400	Requete invalide	Fichier refuse (mauvaise extension, trop lourd...)
404	Introuvable	L'identifiant de tache n'existe pas

6. La migration de base de donnees

La base de donnees de l'application etait deja creee avec l'ancien modele. Quand on ajoute des champs (file_size, content_type) et qu'on modifie audio_file, il faut mettre la base a jour. C'est le role des migrations.

Les deux commandes magiques

```
python manage.py makemigrations
python manage.py migrate
```

makemigrations examine le modele, compare avec la base, et prepare un fichier de migration : une sorte de mode d'emploi pour modifier la base. migrate execute ce mode d'emploi. Dans notre cas, le fichier cree est :

```
transcriptions/migrations/0003_transcriptionjob_content_type_and_more.py
```

Il contient, en langage lisible par Django : "ajouter la colonne content_type, ajouter la colonne file_size, modifier la colonne audio_file". Notre base est maintenant a jour. Les 14 anciennes taches existantes ont ete conservees : la migration ne supprime rien. Leurs champs file_size valent 0, tout simplement parce qu'ils n'avaient pas ete remplis a l'epoque - les nouveaux depots, eux, les remplissent.

Pourquoi c'est mieux que de modifier la base a la main ?

- C'est historique : on peut revenir en arriere (migrate vers une ancienne version).
- C'est reproductible : un collegue clone le projet, lance migrate, et sa base est identique.
- C'est sur : Django calcule lui-meme les operations, sans risque de casser une table.

7. L'interface d'administration

Django fournit gratuitement une interface d'administration, accessible aux super-utilisateurs. Elle permet de voir, chercher et modifier les donnees sans ecrire une ligne de code. On a enregistre notre modele dans `transcriptions/admin.py` :

Ce qu'elle permet de faire

- Voir la liste des taches avec : identifiant court, nom du fichier, statut, taille, langue, date.
- Filtrer par statut (completed, failed...), par langue, par date.
- Rechercher par nom de fichier ou par texte transcrit/corrige.
- Ouvrir une tache pour voir tous ses details (les champs techniques sont en lecture seule).

Y acceder

Lance le serveur, puis ouvre `http://127.0.0.1:8001/admin/` dans ton navigateur et connecte-toi avec un compte super-utilisateur (cree via `python manage.py createsuperuser`).

8. Les tests automatiques

Les tests sont de petits programmes qui verifient que le code fait ce qu'il doit faire. Ils sont essentiels : quand on change quelque chose, on relance les tests, et s'ils passent, on sait qu'on n'a rien casse. Notre fichier `transcriptions/tests.py` contient 37 tests, repartis en 4 familles :

Famille de tests	Nombre	Ce qui est verifie
UploadValidationTest	10	Extension, type MIME et taille acceptes/refuses
TranscriptionJobModelTest	12	Le modele : statuts, compteurs de mots, JSON
APITest	14	Depot, consultation, liste, filtres, suppression
MediaStorageTest	1	Le fichier est bien stocke avec un nom UUID

Exemples de tests concrets

- "Un fichier .txt doit etre refuse" : on poste `document.txt`, on attend un refus (400).
- "Un fichier trop gros doit etre refuse" : on simule une petite limite, on depasse, on attend le refus.
- "Le statut par default est pending" : on cree une tache, on verifie son etat initial.
- "La liste filtrée par `status=completed` ne renvoie que les taches terminees".
- "Le fichier est stocke sous `uploads/<uuid>_<nom>`" : on verifie le chemin sur le disque.

Comment les lancer

```
cd webapp
python manage.py test transcriptions
```

Resultat : Ran 37 tests, OK. C'est la garantie que le socle fonctionne.

9. Mettre en pratique

1. Lancer le serveur

```
cd webapp
python manage.py runserver 127.0.0.1:8001 --noreload
```

Le serveur de developpement démarre et affiche : "Starting development server at http://127.0.0.1:8001/". Il répond désormais sur les adresses du projet.

2. Verifier que l'application répond

```
curl http://127.0.0.1:8001/ # la page accueil (dashboard)
curl http://127.0.0.1:8001/api/stats/ # statistiques en JSON
```

3. Deposer un premier fichier

```
curl -F "audio_file=@/chemin/vers/mon_fichier.wav" \
-F "language=French" http://127.0.0.1:8001/jobs/create/
```

Le serveur répond 201 avec le JSON de la tâche. Note bien son id.

4. Suivre la tâche

```
curl http://127.0.0.1:8001/api/status/<ID_DE_LA_TACHE>/
```

Le status passe de pending à running, puis completed quand la transcription est prête.

5. Par l'interface web

Ouvre <http://127.0.0.1:8001/upload/> dans un navigateur : un formulaire permet de choisir un fichier audio et de le déposer. L'interface affiche aussitôt les erreurs de validation si le fichier est refusé. Toutes les pages restent disponibles : table de bord, liste des tâches, corrections, statistiques.

6. Supprimer

```
curl -X POST http://127.0.0.1:8001/jobs/<ID>/delete-api/
```

10. Resume et prochaines etapes

Ce qui est en place maintenant

- Modele TranscriptionJob enrichi : taille et type de fichier enregistres.
- Stockage sur disque avec nom unique base sur un UUID.
- Validation complete des fichiers : extension, type MIME, taille max (2 Go).
- API JSON : depot (201), consultation, liste filter, statistiques, suppression.
- Migration 0003 appliquee, base a jour, anciennes donnees conservees.
- Administration Django configuree.
- 37 tests automatiques, tous verts.
- Serveur lance sur http://127.0.0.1:8001, verifie avec de vraies reponses.

Idees d'ameliorations pour la suite

- Une file d'attente avec un vrai worker pour traiter plusieurs fichiers en parallele.
- Une authentication pour proteger l'API (les identifiants secrets restent du cote serveur).
- Des entrees d'API paginees et triables, avec une documentation automatique.
- Limiter le nombre ou la taille des depots par utilisateur.
- Un verificateur de contenu (a) ecouter les premieres secondes) pour confirmer qu'il s'agit bien d'un fichier audio.

Annexe. Glossaire du debutant

Terme	Definition simple
API	Ensemble d'adresses web que les programmes utilisent pour echanger des donnees
Base de donnees	Fichier (ou serveur) qui stocke les informations de facon organisee
Champ / colonne	Une information stockee pour chaque ligne (ex: la taille du fichier)
Django	Framework Python pour construire des applications web
Endpoint	Une adresse (URL) precise de l'API, avec son action
JSON	Format de texte lisible qui represente des donnees structurees
Migration	Fichier qui decrit comment faire evoluer la base de donnees
Modèle (model)	Classe Python qui represente une table de la base
MIME	Etiquette technique qui indique le type d'un fichier (audio/mpeg...)
Statut	Etat d'une tache : en attente, en cours, terminee, en erreur
Upload	Action d'envoyer un fichier depuis son ordinateur vers le serveur
UUID	Identifiant quasi-unique (ex: 3f2a9b1e-...) utilise pour nommer les fichiers
Vue (view)	Fonction Python qui repond a une requete web

A retenir : Ce guide a ete genere depuis le projet reel : chaque information correspond a un fichier du codebase. Tu peux ouvrir les fichiers cites (models.py, upload_validation.py, views.py, urls.py, tests.py) pour voir le detail.